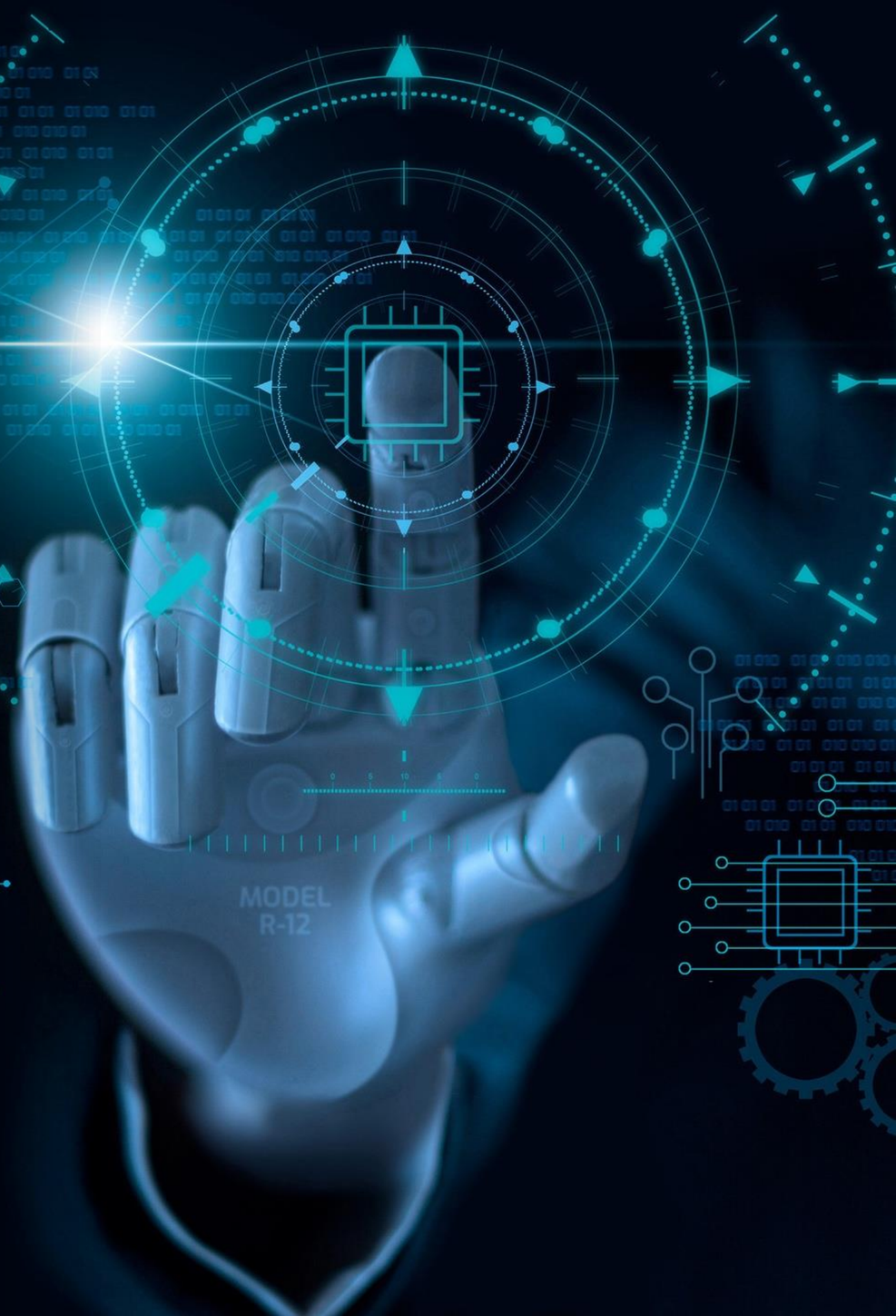
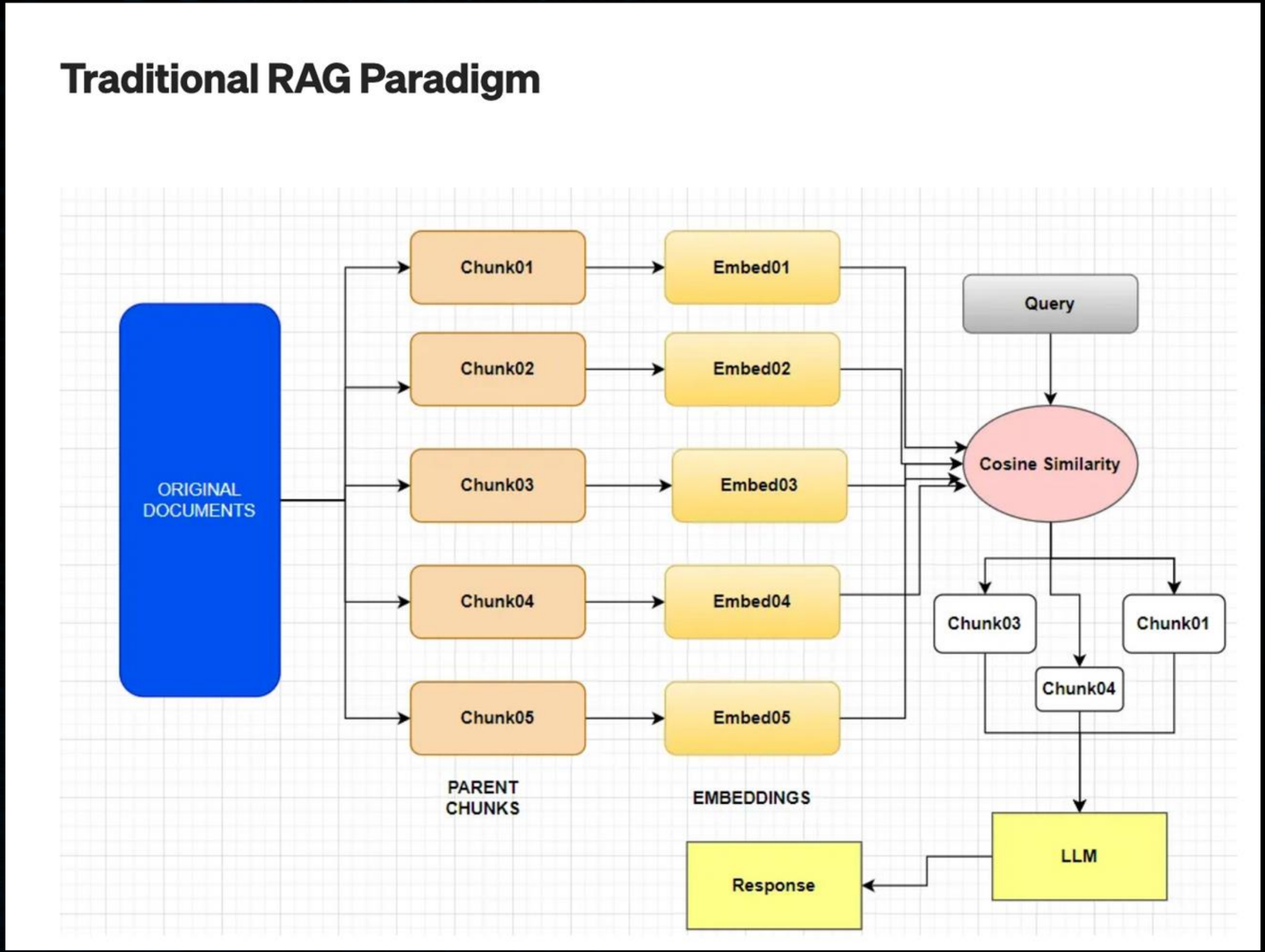
ADVANCE RAG

Presented By **DataSense**



Traditional RAG



Problems with Traditional RAG?

Pre-Retrieval Optimization

1. Enhancing Data Granuality

- **Remove irrelevant text/document:** Eliminated all the irrelevant documents that we don't need LLM to answer. Also remove noise data, this includes removing special characters, stop words (common words like "the" and "a"), and HTML tags.

- **Identify and correct errors:** This includes spelling mistakes, typos, and grammatical errors.

- **Replacing pronouns** with names in split chunks can enhance semantic significance during retrieval.

2. Add MetaData

- Adding metadata, such as concept and level tags, to improve the quality of indexed data.
- Adding metadata information involves integrating referenced metadata, such as dates and purposes, into chunks for filtering purposes, and incorporating metadata like chapters and subsections of references to improve retrieval efficiency.

Here are some scenarios where metadata is useful:

- If you search for items and recency is a criterion, you can sort over a date metadata
- If you search over scientific papers and you know in advance that the information you're looking for is always located in a specific section, say the experiment section for example, you can add the article section as metadata for each chunk and filter on it to match experiments only

Metadata is useful because it brings an additional layer of structured search on top vector search.

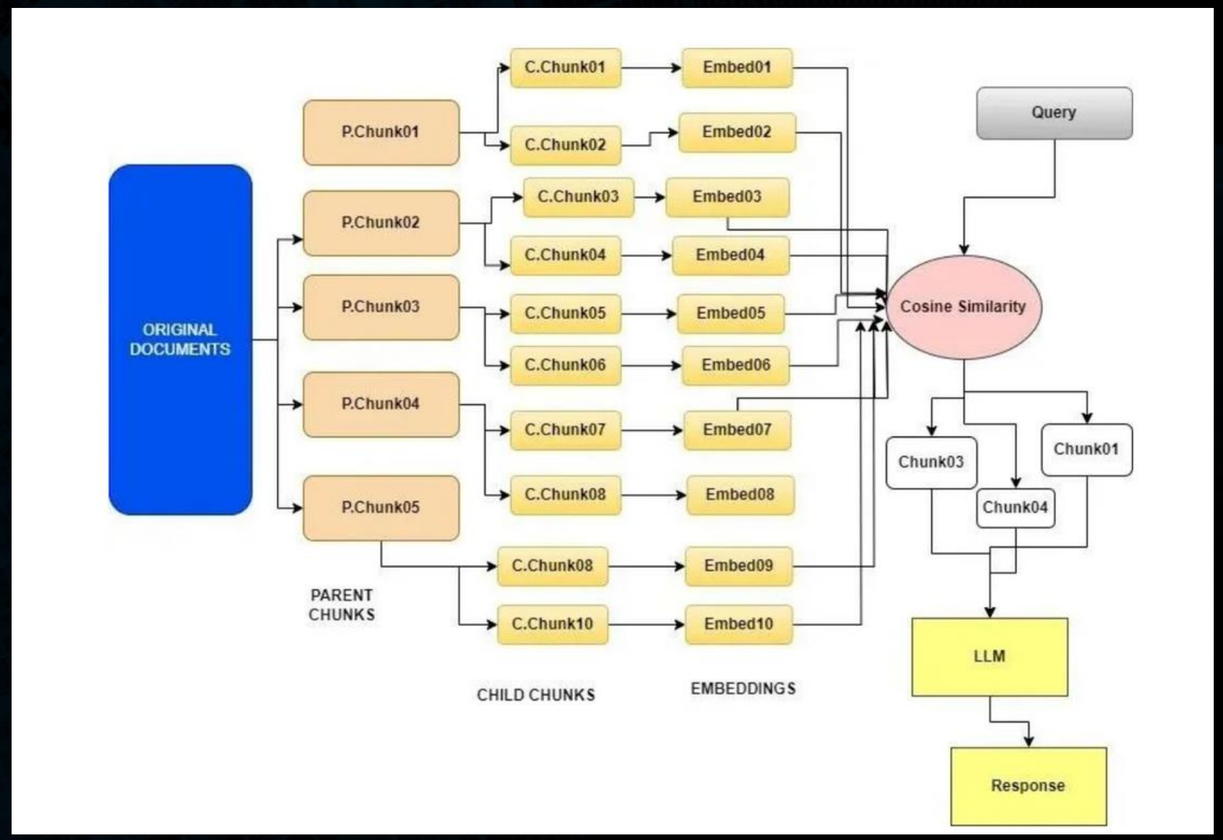
3. Optimizing index structures

- Knowledge Graphs or Graph Neural Network Indexing
- Incorporating information from the graph structure to capture relevant context by leveraging relationships between nodes in a graph data index.

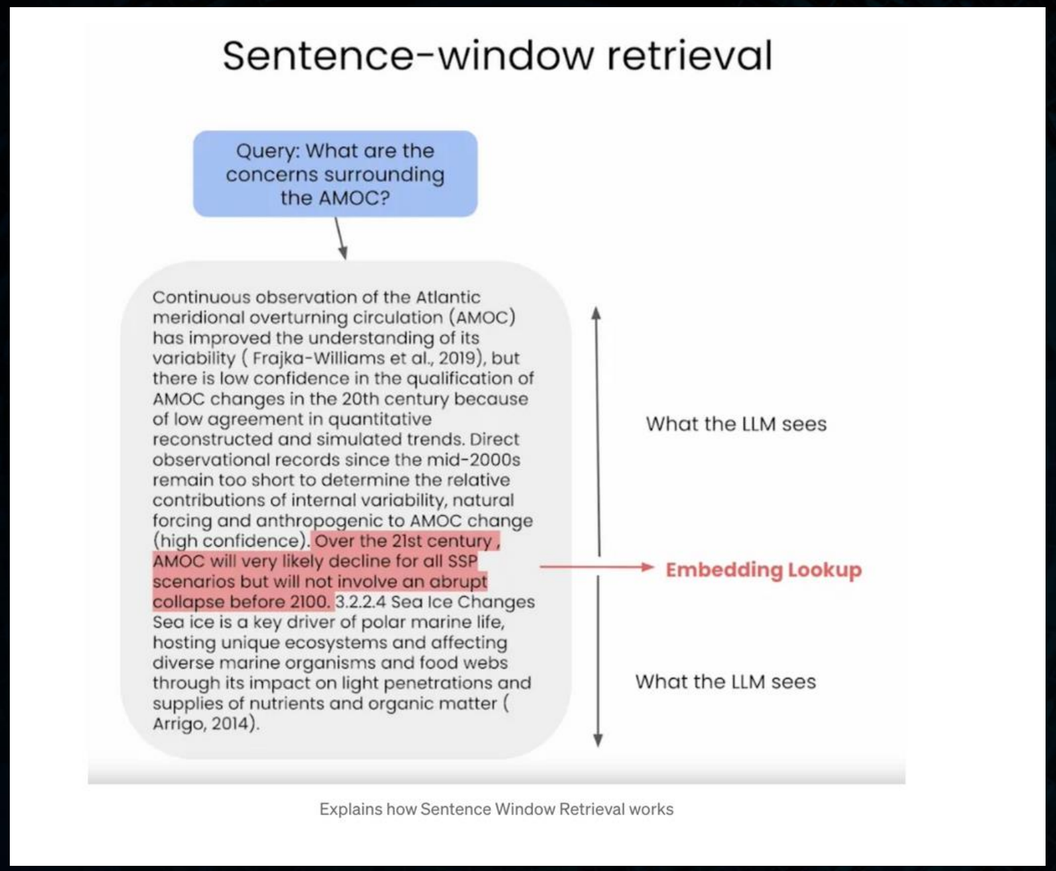
4. Chunking Strategy

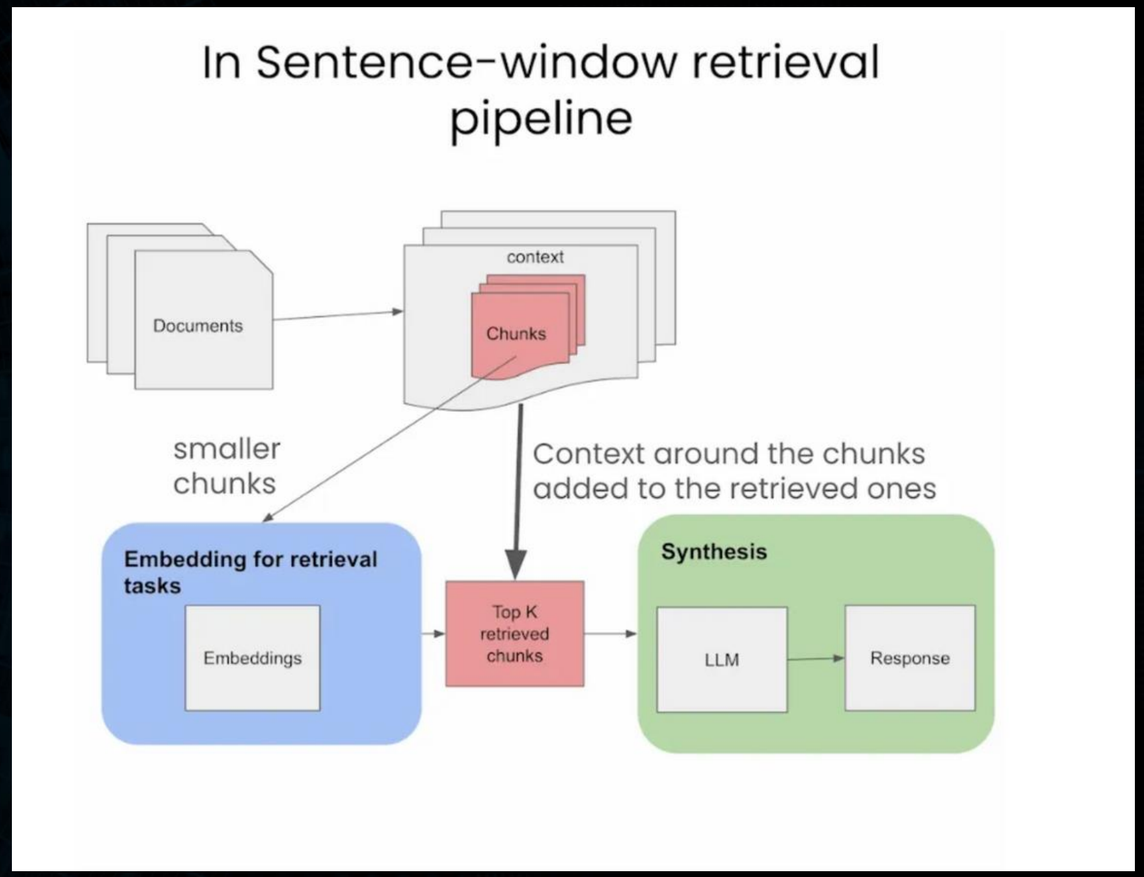
If your chunk is too small, it may not include all the information the LLM needs to answer the user's query; if the chunk is too big, it may contain too much irrelevant information that confuses the LLM or may be too big to fit into the context size.

1. **Parent Document Retriever**



5. Sentence Window Retriever



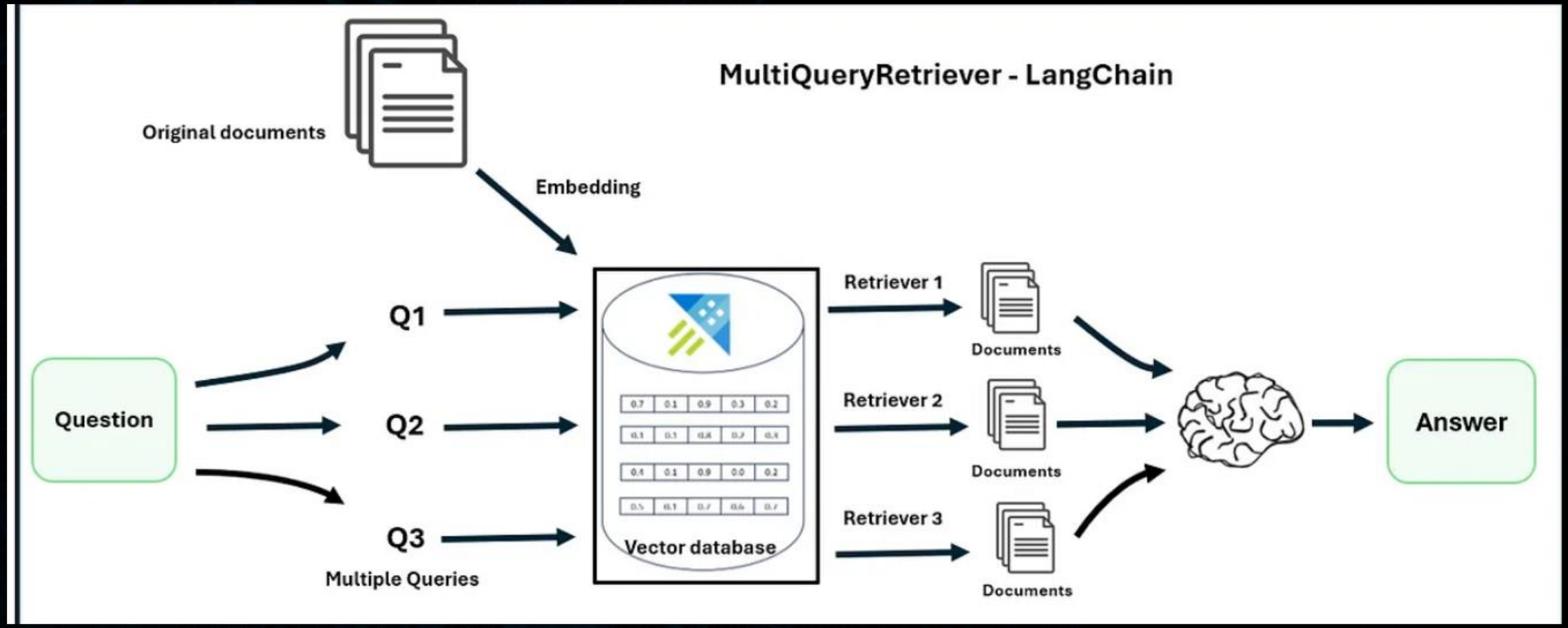


Retrieval Optimization

1. MultiQuery Retrievers- Query Rewriting

- Query rewriting is a fundamental approach for aligning the semantics of a query and a document.
- In this process, we leverage Language Model (LLM) capabilities to rephrase the user's query and give it another shot. It's important to note that two questions that might look the same to a human may not appear similar in the embedding space.

- The Multi-query Retrieval method utilizes LLMs to generate multiple queries from different perspectives for a given user input query, advantageous for addressing complex problems with multiple sub-problems.
- For each query, it retrieves a set of relevant documents and takes the unique union across all queries to get a larger set of potentially relevant documents.
- By generating multiple perspectives on the same question, the MultiQuery Retriever might be able to overcome some of the limitations of the distance-based retrieval and get a richer set of results.


MultiQueryRetriever - LangChain
Original documents
Embedding
Question
Q1
Q2
Q3
Multiple Queries
Vector database
Retriever 1
Retriever 2
Retriever 3
Documents
Documents
Documents
Answer

```
[5] system = """You are a helpful assistant that generates multiple search queries based on a single input query.

    Perform query expansion. If there are multiple common ways of phrasing a user question
    or common synonyms for key words in the question, make sure to return multiple versions
    of the query with the different phrasings.

    If there are acronyms or words you are not familiar with, do not try to rephrase them.

    Return 3 different versions of the question."""

    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", system),
            ("human", "{question}"),
        ]
    )

    chain = prompt | model

    response = chain.invoke({
        "question": "Which food items does this recipe need?"
    })

    response
```

```
AIMessage(content='Here are 3 different versions of the question:\n\n1. What ingredients are required for this recipe?
\n\n2. What are the necessary food components for this dish?\n\n3. Can you list the food items needed to make this
recipe?', response_metadata={'id': 'msg_01TKamMut5Pbg2ihtDwBXJoU', 'model': 'claude-3-5-sonnet-20240620',
'stop_reason': 'end_turn', 'stop_sequence': None, 'usage': {'input_tokens': 115, 'output_tokens': 53}}, id='run-
a63bf24c-6e3e-4ebb-b808-07db8855f476-0', usage_metadata={'input_tokens': 115, 'output_tokens': 53, 'total_tokens':
168})
```

2. Sub-queries

If the user query contains multiple questions, this can make context retrieval tricky. Each question probably needs different information, and we are not going to get all of it using all the questions as basis for information retrieval. To solve this problem, we can decompose the input into multiple sub-queries, and perform retrieval for each of the sub-queries.

```python
system_decompose = """You are a helpful assistant that generates search queries based on a single input query.

Perform query decomposition. Given a user question, break it down into distinct sub questions that
you need to answer in order to answer the original question.

If there are acronyms or words you are not familiar with, do not try to rephrase them."""

prompt = ChatPromptTemplate.from_messages(
    [
        ("system", system_decompose),
        ("human", "{question}"),
    ]
)

chain = prompt | model

response = chain.invoke({
    "question": """Which is the most popular programming language for machine learning and
is it the most popular programming language overall?"""
})

response
```

```
AIMessage(content='To answer this question, we need to break it down into the following sub-questions:\n\n1. What are the most popular programming languages for machine learning?\n2. Which one among these is considered the most popular for machine learning?\n3. What are the most popular programming languages overall?\n4. How does the most popular language for machine learning compare to the most popular language overall?', response_metadata={'id': 'msg_015ua5qkd5APXbAU8APdgys1', 'model': 'claude-3-5-sonnet-20240620', 'stop_reason': 'end_turn', 'stop_sequence': None, 'usage': {'input_tokens': 101, 'output_tokens': 83}}, id='run-b13038bb-3820-4470-b00b-ea6773deb07c-0', usage_metadata={'input_tokens': 101, 'output_tokens': 83, 'total_tokens': 184})
```

3. STEP Back Prompt

Many questions can be a bit too complex for the RAG pipeline's retrieval to grasp the multiple levels of information needed to answer them. For these cases, it can be helpful to generate multiple additional queries to use for retrieval. These queries will be more generic than the original query. This will enable the RAG pipeline to retrieve relevant information on multiple levels.

```
[7] system_step_back = """You are an expert at taking a specific question and extracting a more generic question that gets at
    the underlying principles needed to answer the specific question.

    Given a specific user question, write a more generic question that needs to be answered in order to answer the specific question.

    If you don't recognize a word or acronym to not try to rewrite it.

    Write concise questions."""

    prompt = ChatPromptTemplate.from_messages(
        [
            ("system", system_step_back),
            ("human", "{question}"),
        ]
    )

    chain = prompt | model

    response = chain.invoke({
        "question": """Which is the most popular programming language for machine learning?"""
    })

    response
```

```
AIMessage(content='What are the most commonly used programming languages in the field of artificial intelligence and data science?',
response_metadata={'id': 'msg_012CxLK8dgbkWMbNm8e5xhBQ', 'model': 'claude-3-5-sonnet-20240620', 'stop_reason': 'end_turn', 'stop_sequence':
None, 'usage': {'input_tokens': 98, 'output_tokens': 21}}, id='run-827d9a2d-2825-4138-bf51-5b055313ade8-0', usage_metadata={'input_tokens':
98, 'output_tokens': 21, 'total_tokens': 119})
```

4. HYDE- Hypothetical Document Embeddings

Both Hyde and Query2doc are similar query rewriting optimisations. Given that search queries are often short, ambiguous, or lack necessary background information, LLMs can provide relevant information to guide retrieval systems, as they memorize an enormous amount of knowledge and language patterns by pre-training on trillions of tokens.

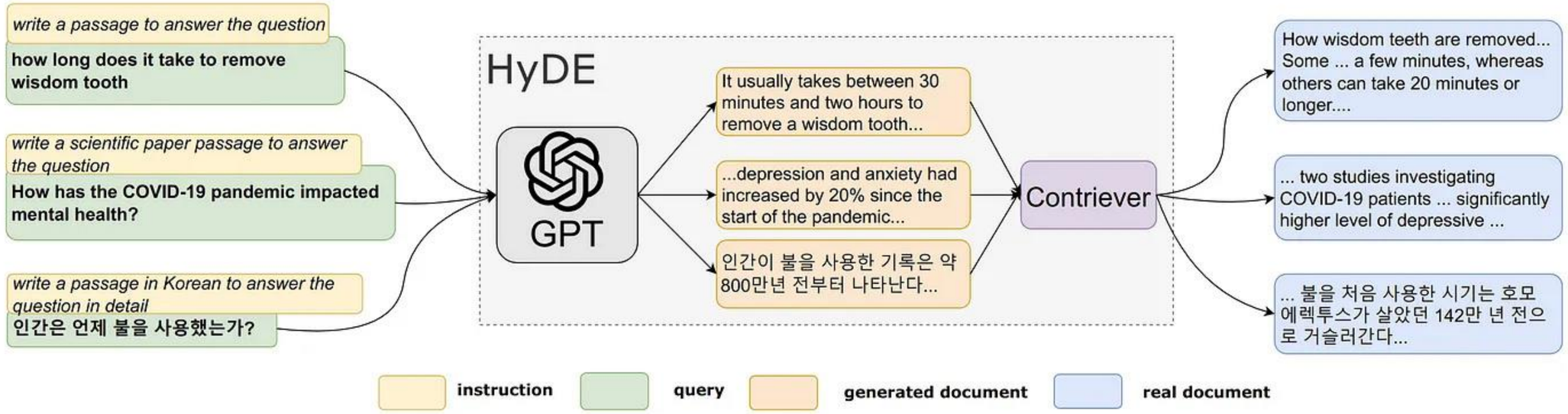
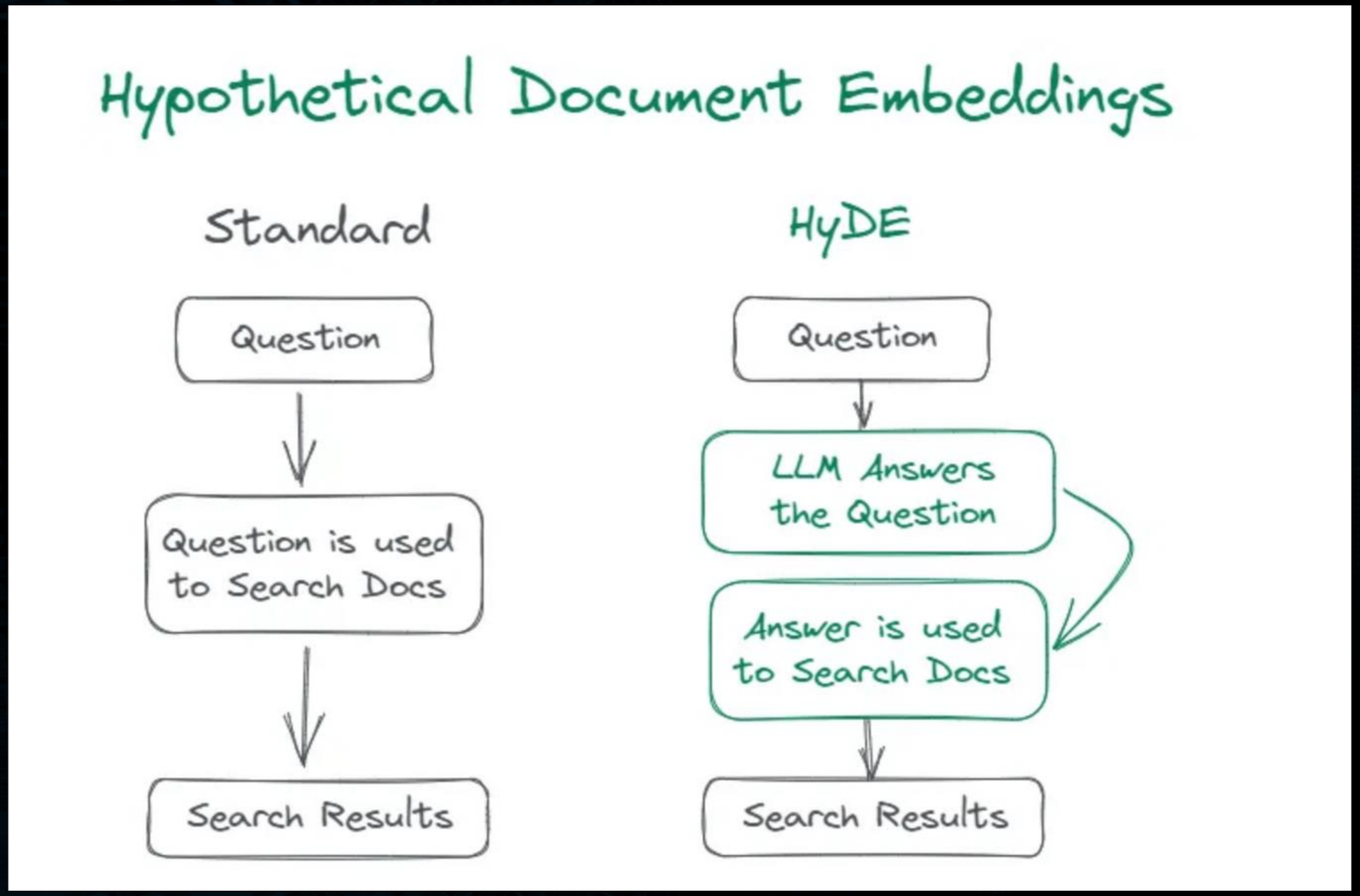


Figure 1: An illustration of the HyDE model. Documents snippets are shown. HyDE serves all types of queries without changing the underlying GPT-3 and Contriever/mContriever models.

4. HYDE- Hypothetical Document Embeddings



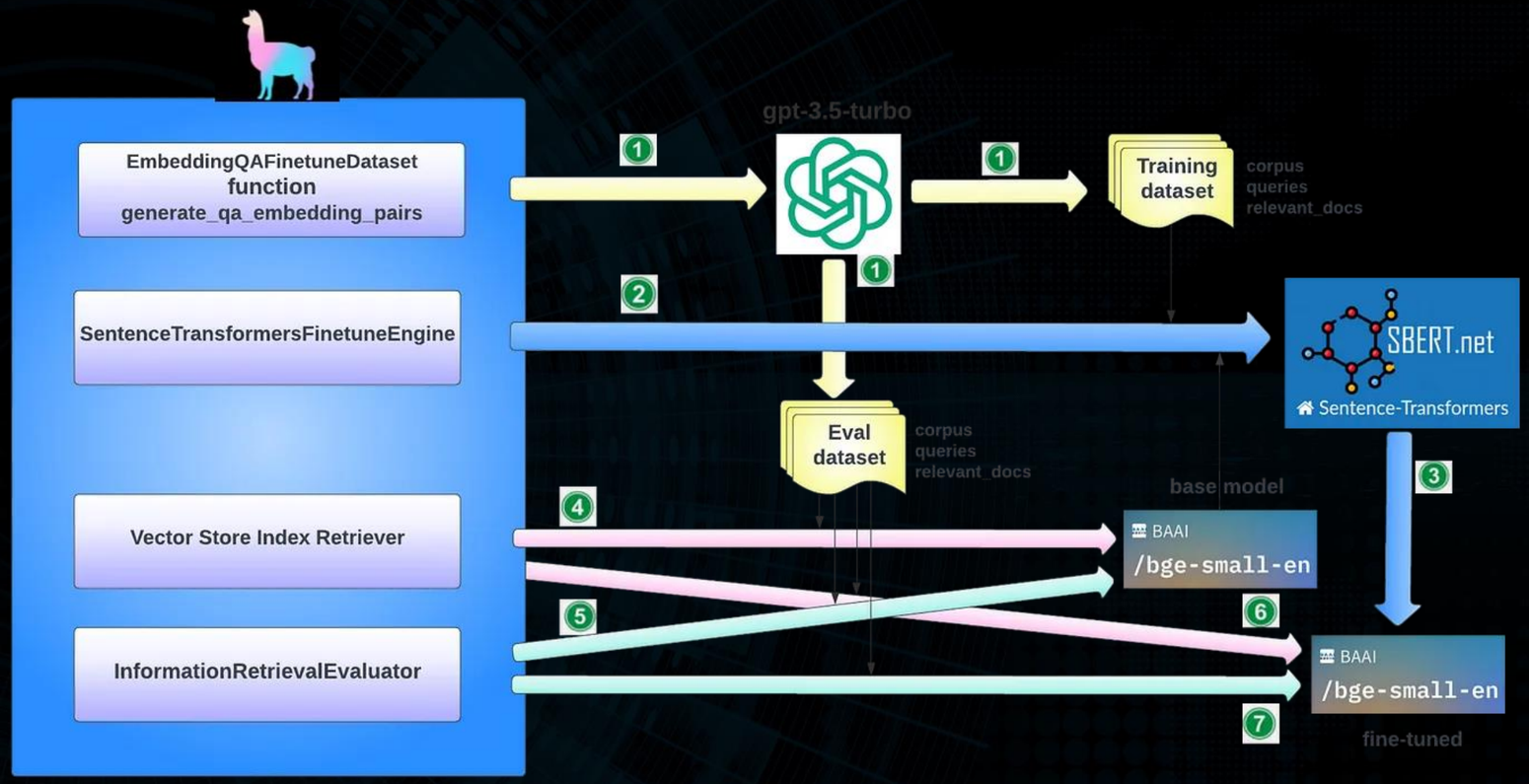
5. Fine Tune Embedding

Fine-tuning embedding models significantly impact the relevance of retrieved content in RAG systems. This process involves customizing embedding models to enhance retrieval relevance in domain-specific contexts, especially for professional domains dealing with evolving or rare terms.

Generating synthetic dataset for training and evaluation

The key idea here is that training data for fine-tuning can be generated using language models like GPT-3.5-turbo to formulate questions grounded on document chunks. This allows us to generate synthetic positive pairs of (query, relevant documents) in a scalable way without requiring human labellers. Final dataset will be pairs of questions and text chunks.

5. Fine Tune Embedding



6. Hybrid Search

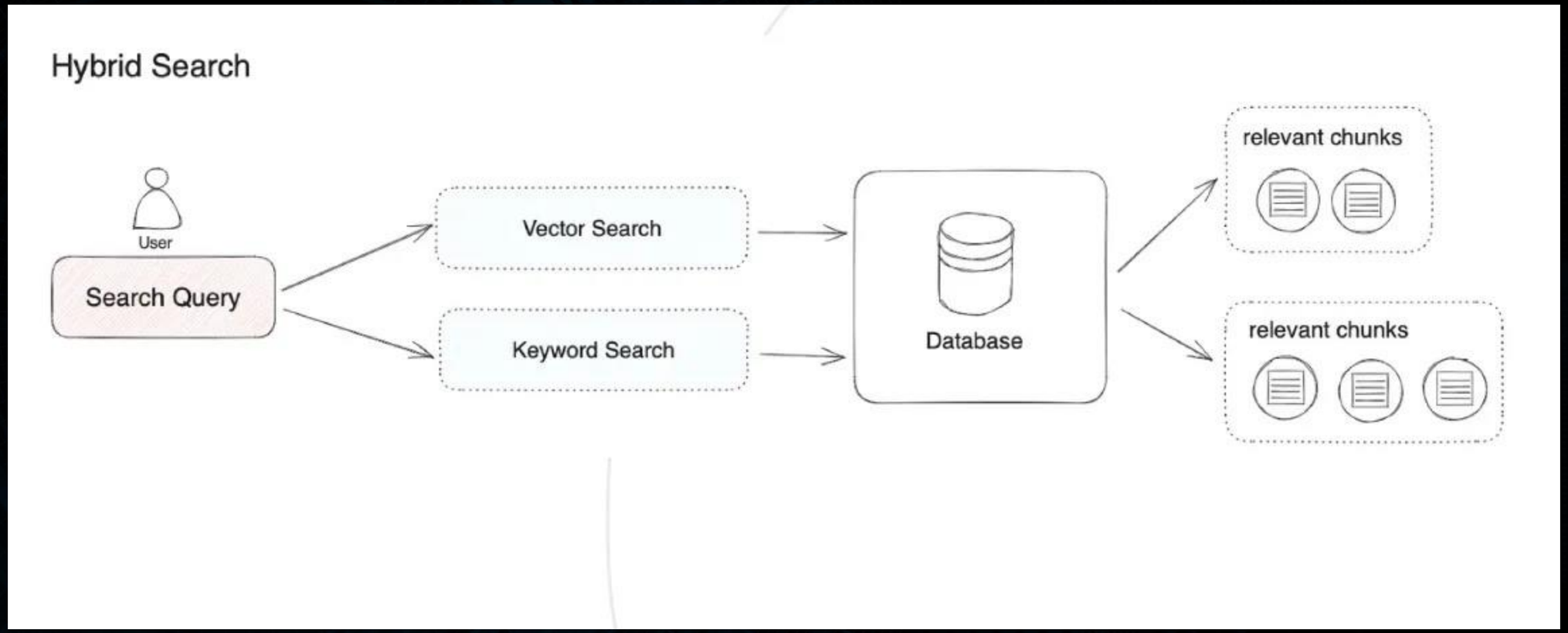
The RAG system optimizes its performance by intelligently integrating various techniques, including keyword based search, semantic search, and vector search.

This approach leverages the unique strengths of each method to accommodate diverse query types and information needs, ensuring consistent retrieval of highly relevant and context-rich information. The use of hybrid search serves as a robust supplement to retrieval strategies, thereby enhancing the overall efficacy of the RAG pipeline.

Common Example

The most common pattern is to combine a sparse retriever (like BM25) with a dense retriever (like embedding similarity), because their strengths are complementary. It is also known as “hybrid search”. The sparse retriever is good at finding relevant documents based on keywords, while the dense retriever is good at finding relevant documents based on semantic similarity.

6. Hybrid Search



Post Retrieval Optimization

1. Re-Ranking

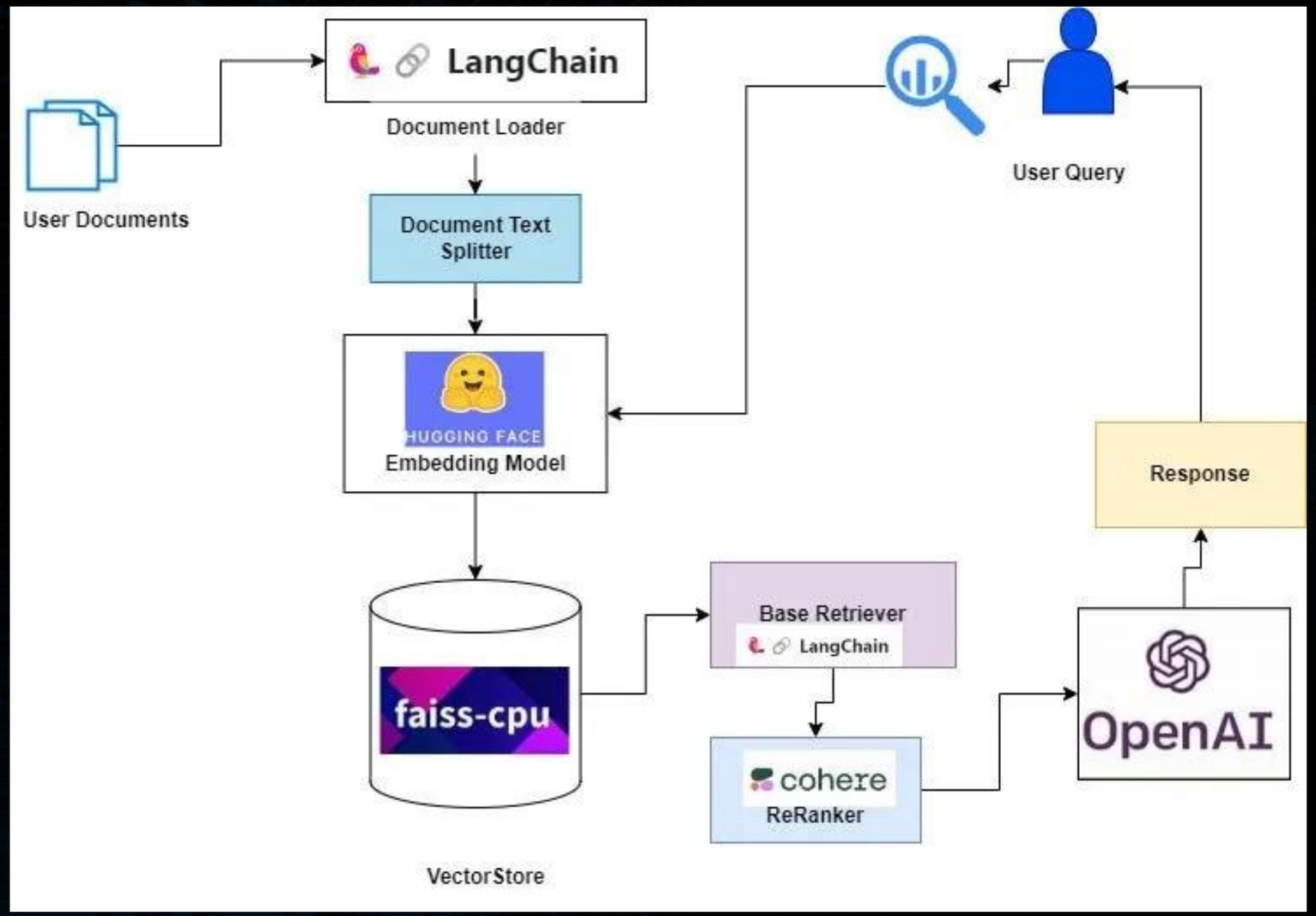
Reranking retrieval results before sending them to the LLM has significantly improved RAG performance.

A high score in vector similarity search does not mean that it will always have the highest relevance.

The core concept involves re-arranging document records to prioritize the most relevant items at the top, thereby limiting the total number of documents. This not only resolves the challenge of context window expansion during retrieval but also enhances retrieval efficiency and responsiveness.

Increase the similarity_top_k in the query engine to retrieve more context passages, which can be reduced to top_n after reranking.

1. Re-Ranking


LangChain
Document Loader
User Documents
Document Text Splitter
HUGGING FACE
Embedding Model
User Query
Response
faiss-cpu
VectorStore
Base Retriever
LangChain
cohere
ReRanker
OpenAI

2. Prompt Compression

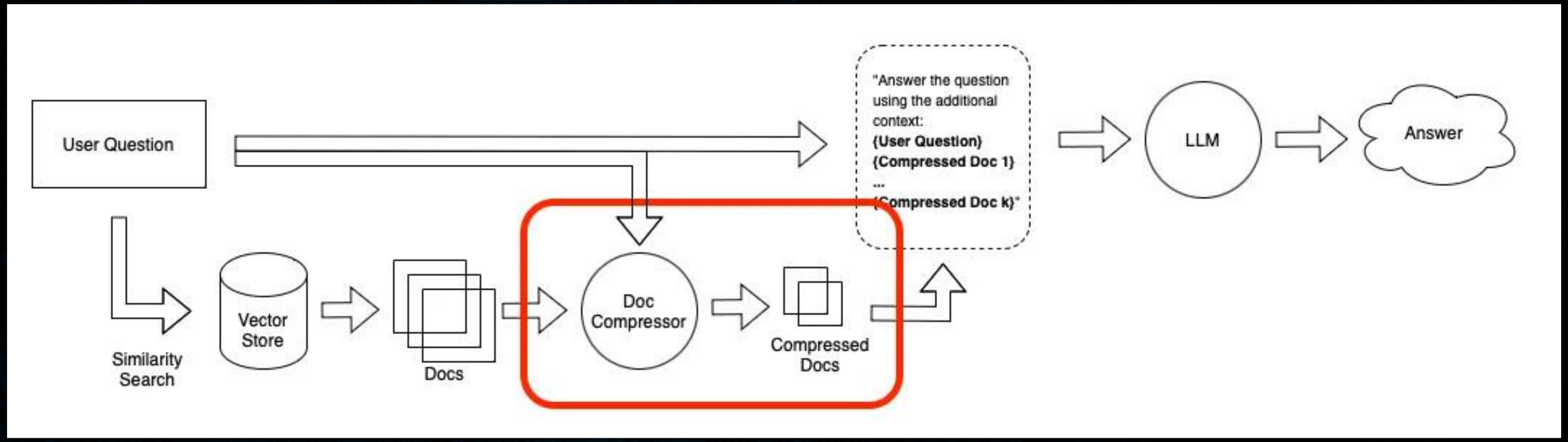
Noise in retrieved documents adversely affects RAG performance therefore, information most relevant to a query may be buried in a document with a lot of irrelevant text. Passing that full document through your application can lead to more expensive LLM calls and poorer responses.
Here, the emphasis lies in compressing irrelevant context, highlighting pivotal paragraphs, and reducing the overall context length.

Contextual compression

Contextual compression is meant to fix this. The idea is simple: instead of immediately returning retrieved documents as-is, it can compress them using the context of the given query, so that only the relevant information is returned. “Compressing” here refers to both compressing the contents of an individual document and filtering out documents wholesale.

Doc Compressor is a small language models to calculate prompt mutual information of user query and retrieved document, estimating element importance.

2. Prompt Compression



Thank You

Presented By **DataSense**